

Discente: Ronaldo Ribeiro Porto Filho – 202410131

Data: 28/09/2025

Exercício complexidade de tempo do algoritmo Heapsort

Vamos analisar a complexidade de tempo do algoritmo Heapsort, o qual representa a notação Big-O do mesmo.

```
HeapSort(v){  
    Build(v);  
    for(i = n; i > 1; i--){  
        Troque v[i] com v[1]  
        Sift(1, i-1);  
    }  
}
```

Primeiro, a função Build tem custo linear, pois quando somamos todos os ajustes das funções Sift presentes na mesma, que possuem custo logarítmico, o total é limitado por uma série que quando resolvida o resultado é $O(n)$.

Com isso, o laço for vai percorrer o vetor da última posição até o segundo elemento (índice 1), trocando o primeiro elemento com o último daquela iteração, e chamando a função Sift após cada troca, para a estrutura do Heap não se quebrar.

Dessa forma, o laço será executado $n - 1$ vezes, chamando a função Sift a cada execução, e como já foi mostrado, esta função tem custo logarítmico, ou seja, quando juntamos o tempo do Build com quantas vezes o Sift é executado, temos:

$$T(n) = O(n) + (n - 1) * O(\log n), n > 1$$

$$T(n) = n + (n - 1) * \log n, n > 1$$

$$T(n) = n + n \log n - \log n, n > 1$$

Termo de maior ordem: $n \log n$.

$$T(n) \in O(n \log n)$$